

Politechnika Świętokrzyska w Kielcach

Wydział Elektrotechniki, Automatyki i Informatyki



Politechnika Świętokrzyska
Kielce University of Technology

Technologie Obiektowe

Projekt

Wsparcie dla dokumentów json w składach / bazach danych aplikacja webowa ToDo

Skład zespołu:

Beniamin Żak

Jakub Maliński

Kierunek/specjalność: Informatyka/Systemy Informacyjne

Studia: stacjonarne II Stopnia

Numer grupy: 1ID21B

Wstęp teoretyczny

Współczesne aplikacje coraz częściej przetwarzają dane o zmiennej, rozbudowanej lub częściowo nieregularnej strukturze. Tradycyjny model relacyjny, oparty na tabelach, kolumnach, kluczach głównych i obcych, bardzo dobrze sprawdza się w systemach wymagających wysokiej spójności danych, jasno określonych relacji oraz złożonych zapytań SQL. Jednak w wielu przypadkach dane wykorzystywane przez aplikacje mają postać dokumentów, konfiguracji, obiektów API lub struktur zagnieżdżonych, które można zapisać w formacie JSON.

Celem projektu jest stworzenie aplikacji w celu analizy wsparcia dla dokumentów JSON w składach danych oraz bazach danych. W pracy porównano dwa podejścia: relacyjna baza danych PostgreSQL, która umożliwia przechowywanie danych JSON i JSONB, oraz dokumentowa baza MongoDB, która od początku została zaprojektowana do pracy z dokumentami podobnymi do JSON.

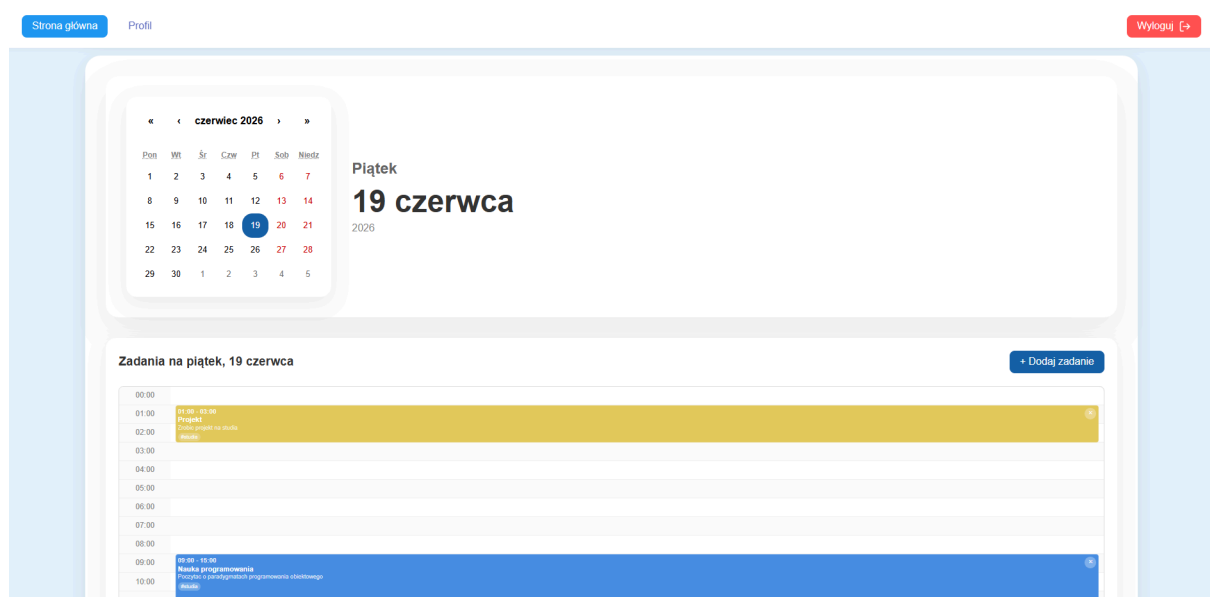
Wybór odpowiedniego sposobu przechowywania danych wpływa na późniejszą wydajność aplikacji, prostotę zapytań, łatwość rozwoju systemu oraz bezpieczeństwo i poprawność danych. W projekcie przedstawiono różnice między podejściem relacyjnym a dokumentowym, a także testy pozwalające ocenić, w jakich sytuacjach lepiej sprawdza się PostgreSQL z obsługą JSONB, a w jakich MongoDB jako skład dokumentów.

Technologie:

- JavaScript - główny język programowania
- MongoDB - baza w składach dokumentów
- PostgreSQL - baza relacyjna
- React - framework wykorzystywany na Frontendzie
- JSON - format danych do przesyłania informacji

Interfejs graficzny

- Strona Główna:



Rysunek 1. Ekran strony głównej

- Dodawanie/edycja zadania:

Zadania na piątek, 19 czerwca Zamknij

Edytuj zadanie

Od: 09:00

Do: 15:00

Tytuł: Nauka programowania

Opis: Poczytać o paradygmatach programowania obiektowego

Tagi: studia

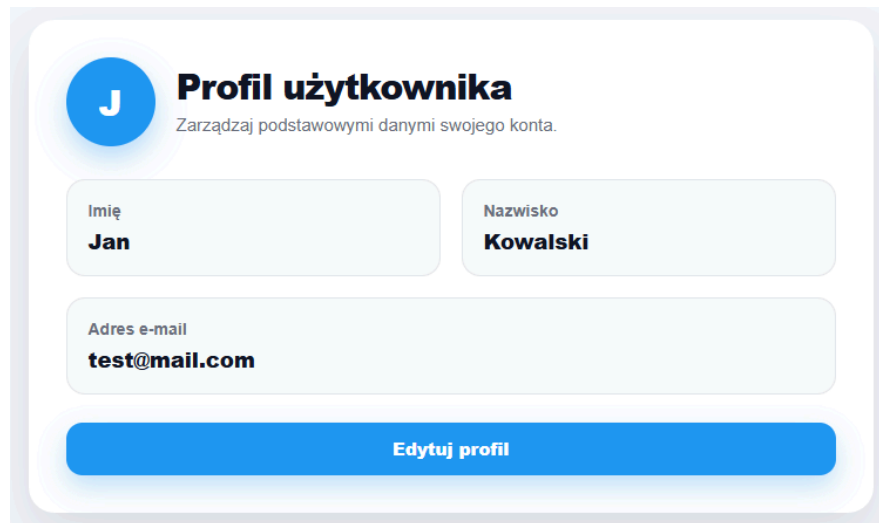
Wpisz tagi po przecinku. Maksymalnie zostanie zapisanych 10 tagów.

Kolor: blue red green yellow purple

Zapisz zmiany Anuluj

Rysunek 2. Okno dodawania lub edycji zadania

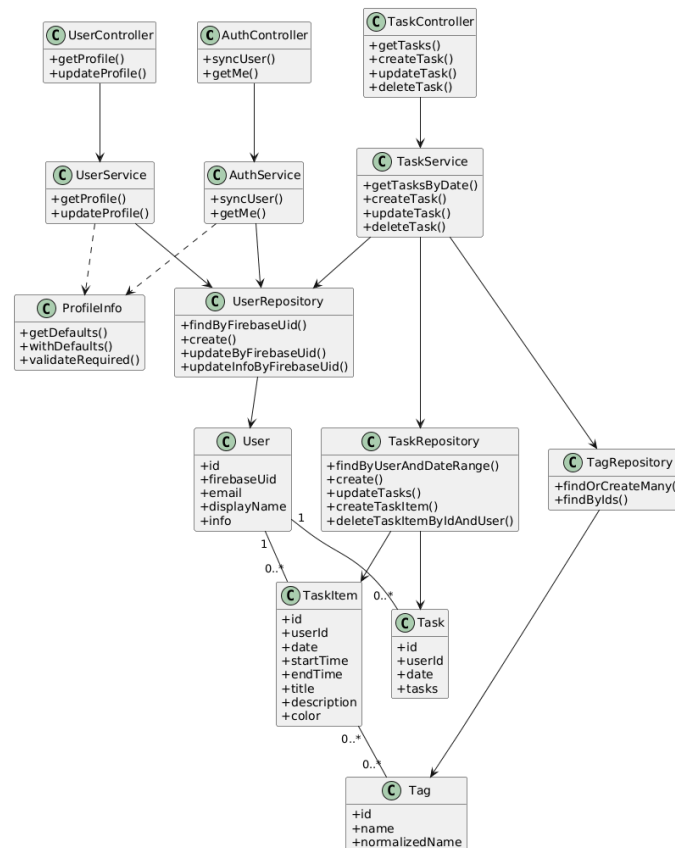
- Ekran profilu:



Rysunek 3. Ekran profilu

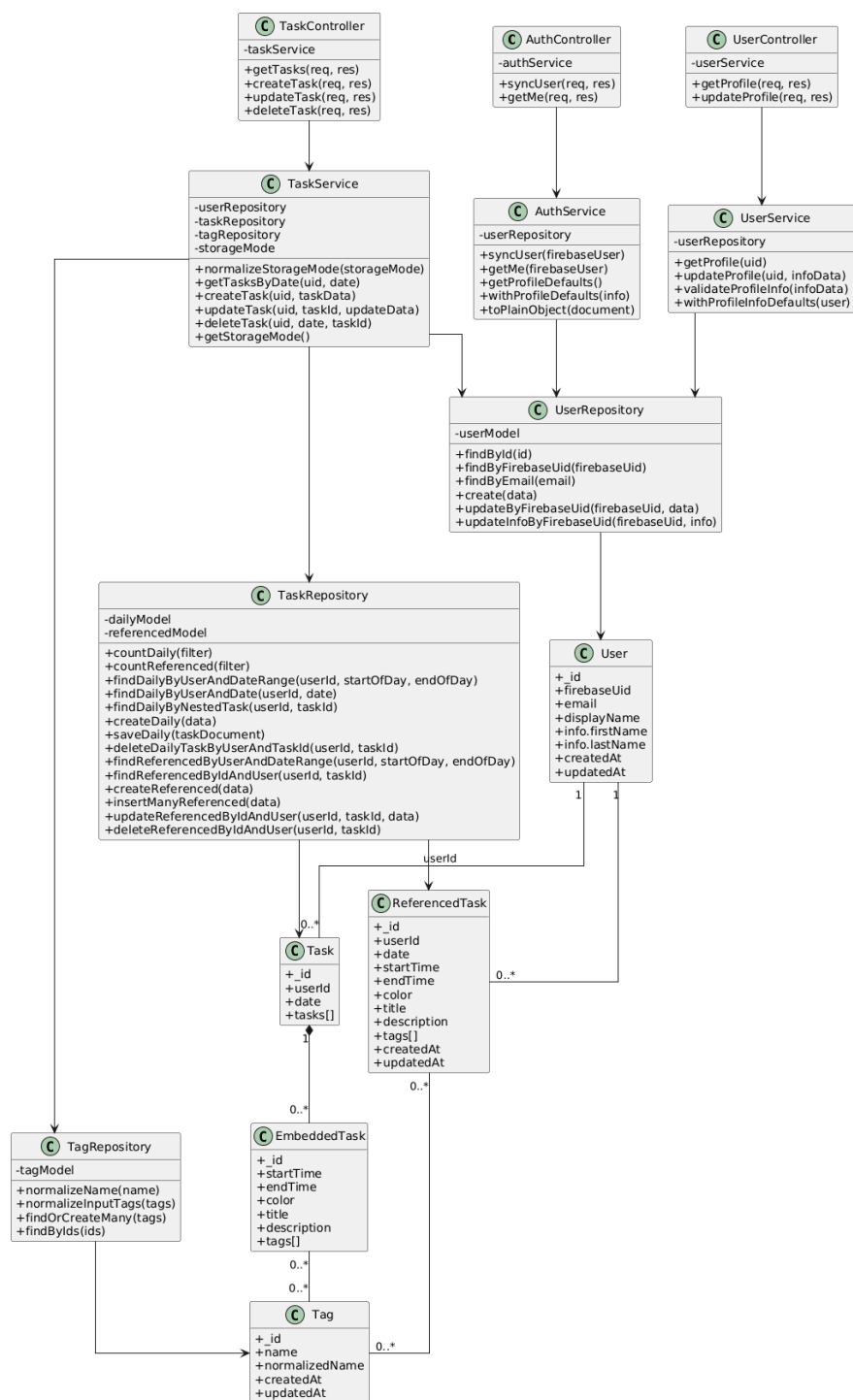
Diagramy klas

Backend PostgreSQL:



Rysunek 4. Diagram klas dla backendu PostgreSQL

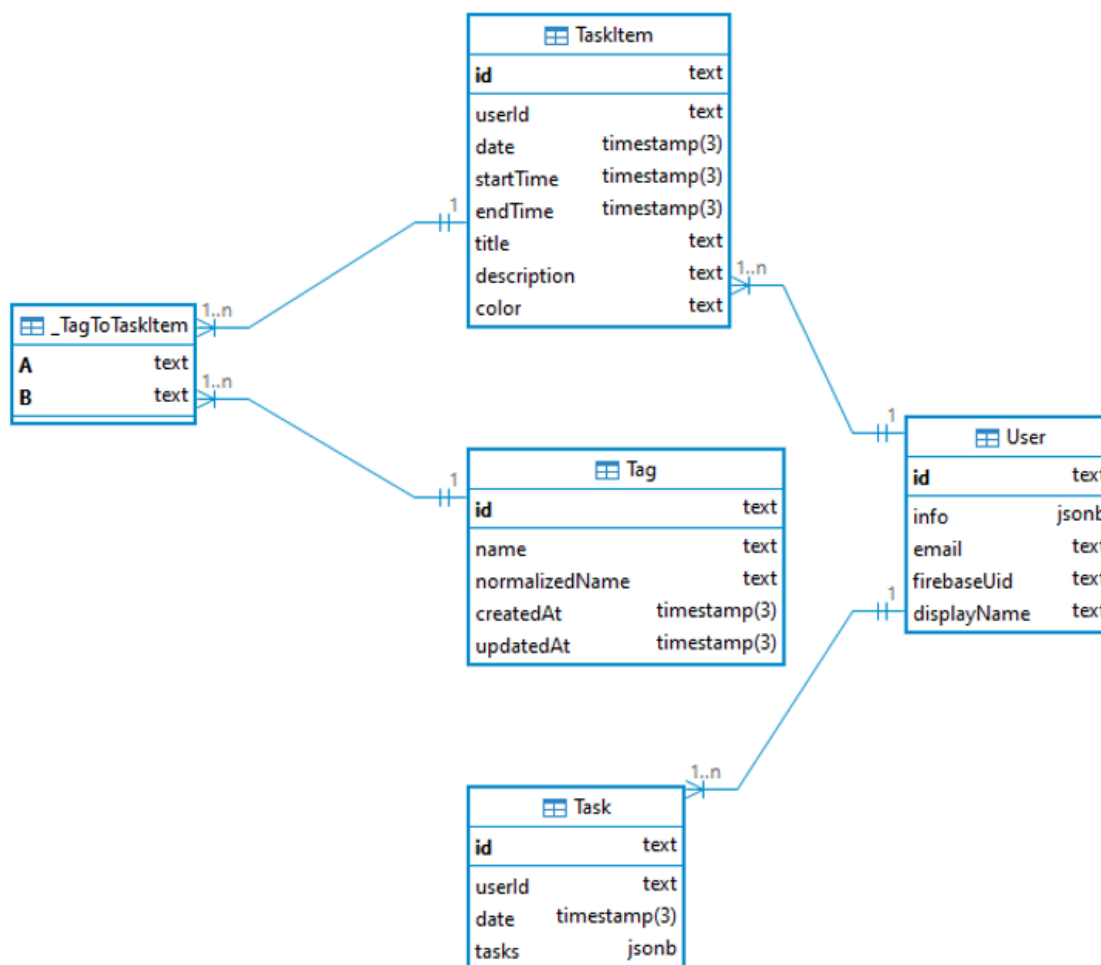
Backend MongoDB:



Rysunek 5. Diagram klas dla backendu MongoDB

Diagramy związków encji

PostgreSQL:



Rysunek 6. Diagram ERD dla PostgreSQL

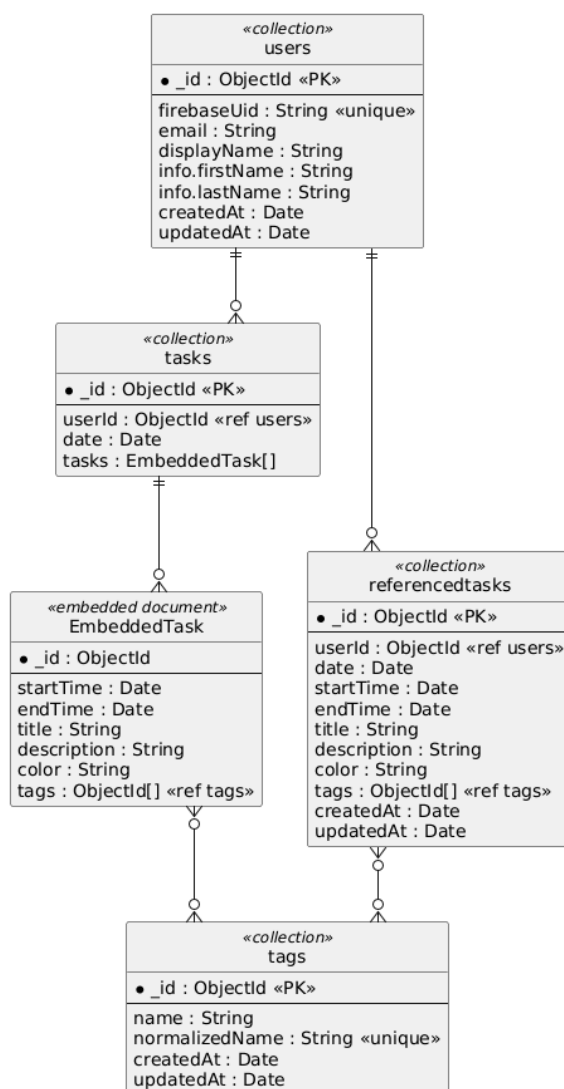
Główną tabelą jest tabela *User*, która przechowuje dane użytkowników, takie jak adres e-mail, wyświetlana nazwa oraz identyfikator Firebase. Z użytkownikiem powiązane są tabele *TaskItem*, *Task* w relacji jeden-do-wielu, co oznacza, że jeden użytkownik może mieć przypisanych wiele zadań.

Tabela *TaskItem* przechowuje pojedyncze zadania wraz z datą, godziną rozpoczęcia i zakończenia, tytułem, opisem oraz kolorem.

Tabela *Tag* przechowuje tagi tworzone przez użytkownika. Relacja wiele-do-wielu pomiędzy zadaniami *TaskItem* a tagami *Tag* została zrealizowana za pomocą tabeli pośredniej *_TagToTaskItem*, która łączy identyfikatory obu tabel.

Dodatkowo tabela *Task* przechowuje dane zadań w formacie jsonb, co pozwala zapisywać bardziej złożone struktury danych. Diagram pokazuje klucze główne, pola tabel oraz relacje między encjami w bazie PostgreSQL.

MongoDB:



Rysunek 7. Diagram ERD dla MongoDB

Główną kolekcją jest *users*, która przechowuje dane użytkowników, takie jak identyfikator, e-mail, nazwa wyświetlana, imię, nazwisko oraz daty utworzenia i aktualizacji.

Z użytkownikami powiązane są zadania. Na diagramie pokazano dwa sposoby ich przechowywania. Pierwszy to model *embedded*, w którym zadania są zapisane jako dokumenty zagnieżdżone *EmbeddedTask* wewnątrz dokumentu *tasks*. Drugi to model *referenced*, w którym każde zadanie znajduje się w osobnej kolekcji *referencedtasks* i odwołuje się do użytkownika przez pole *userId*.

Kolekcja *tags* przechowuje tagi przypisywane do zadań. Zarówno zadania zagnieżdżone, jak i referencyjne mogą posiadać wiele tagów, dlatego powiązanie z tagami ma charakter wiele-do-wielu.

W MongoDB pola oznaczone jako *ref users* lub *ref tags* nie są klasycznymi kluczami obcymi jak w PostgreSQL. Są to jedynie identyfikatory dokumentów, dlatego poprawność takich powiązań musi być kontrolowana głównie przez aplikację.

Najważniejsze fragmenty kodu

1. Inicjalizacja firebase admin SDK

```
const admin = require("firebase-admin");

const serviceAccount = require("../serviceAccountKey.json");

if (!admin.apps.length) {
  admin.initializeApp({
    credential: admin.credential.cert(serviceAccount),
  });
}

module.exports = admin;
```

Powyższy kod odpowiada za inicjalizację Firebase Admin SDK po stronie backendu. Dzięki temu serwer może korzystać z usług Firebase, przede wszystkim do weryfikacji tokenów użytkowników. Warunek *if (!admin.apps.length)* zabezpiecza aplikację przed wielokrotną inicjalizacją Firebase.

2. Weryfikacja tokenu firebase

```
const decodedToken = await admin.auth().verifyIdToken(token);

req.firebaseUser = decodedToken;

next();
```

Powyższy kod przedstawia najważniejszy fragment middleware odpowiedzialnego za autoryzację użytkownika. Token przesłany w nagłówku żądania jest sprawdzany przez Firebase. Jeżeli token jest poprawny, dane użytkownika zostają zapisane w *req.firebaseUser*, dzięki czemu kolejne części backendu mogą rozpoznać zalogowanego użytkownika.

3. Przechowywanie zadań jako JSON w PostgreSQL

```
model Task {
  id      String  @id @default(uuid())
  userId  String
  date    DateTime
  tasks   Json
  user    User @relation(fields: [userId], references: [id], onDelete: Cascade) }
```

Powyższy kod przedstawia model Task w PostgreSQL. Pole tasks typu Json pozwala przechowywać listę zadań w formie dokumentu JSON. Jest to wariant wykorzystywany do analizy przechowywania danych częściowo zagnieżdżonych w relacyjnej bazie danych.

4. Relacyjny model przechowywania zadań w PostgreSQL

```
model TaskItem {
  id          String  @id @default(uuid())
  userId      String
  date        DateTime
  startTime   DateTime
  endTime     DateTime
  title       String  @default("")
  description  String  @default("")
  color       String  @default("#4A90E2")

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
  tags Tag[]
}
```

Powyższy kod opisuje alternatywny, relacyjny sposób przechowywania zadań. Każde zadanie jest osobnym rekordem w tabeli TaskItem, a relacja z użytkownikiem oraz tagami jest wyrażona za pomocą powiązań między tabelami. Ten wariant umożliwia porównanie klasycznego modelu relacyjnego z przechowywaniem danych jako JSON.

5. Dokumentowy model zadań w MongoDB

```
const dailyTaskSchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  date: { type: Date, required: true },
  tasks: [
    {
      startTime: Date,
      endTime: Date,
      color: String,
```

```

    title: String,
    description: String,
    tags: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Tag' }]
  }
]
});

```

Powyższy kod przedstawia dokumentowy sposób przechowywania zadań w MongoDB. Zadania znajdują się jako tablica obiektów wewnątrz jednego dokumentu dziennego. Takie rozwiązanie dobrze odzwierciedla naturalny sposób pracy MongoDB z dokumentami podobnymi do JSON.

6. Referencyjny model zadań w MongoDB

```

const referencedTaskSchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  date: Date,
  startTime: Date,
  endTime: Date,
  color: String,
  title: String,
  description: String,
  tags: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Tag' }]
});

```

Powyższy kod przedstawia drugi wariant przechowywania zadań w MongoDB. Każde zadanie jest osobnym dokumentem, który posiada referencję do użytkownika oraz tagów. Pozwala to porównać dokument osadzony z podejściem opartym na referencjach.

7. Wybór sposobu przechowywania danych

```

async getTasksByDate(uid, date) {
  const user = await this.findUser(uid);

  if (this.storageMode === TASK_STORAGE_MODES.RELATIONAL) {
    return this.getRelationalTasksByDate(user.id, date);
  }
}

```

```

}

return this.getJsonbTasksByDate(user.id, date);
}

```

Powyższy kod opisuje mechanizm wyboru sposobu pobierania danych. W zależności od wartości `TASK_STORAGE_MODE` aplikacja korzysta z wariantu relacyjnego albo z wariantu JSONB. Dzięki temu można porównywać różne sposoby przechowywania danych bez zmiany działania interfejsu aplikacji.

8. Tworzenie zadania w wybranym trybie przechowywania

```

async createTask(uid, taskData) {
  const user = await this.findUser(uid);

  if (this.storageMode === TASK_STORAGE_MODES.RELATIONAL) {
    return this.createRelationalTask(user.id, taskData);
  }

  return this.createJsonbTask(user.id, taskData);
}

```

Powyższy kod przedstawia analogiczny mechanizm dla operacji tworzenia zadania. Serwis najpierw identyfikuje użytkownika, a następnie zapisuje dane w odpowiednim wariantcie przechowywania.

9. Pomiar czasu wykonania zapytań

```

class DatabaseQueryTimer {
  static async measure(operationName, queryCallback) {
    const start = process.hrtime.bigint();

    try {
      return await queryCallback();
    } finally {
      const end = process.hrtime.bigint();
    }
  }
}

```

```

const durationMs = Number(end - start) / 1_000_000;

console.log(
  `[PostgreSQL][DB] ${operationName} - ${durationMs.toFixed(2)} ms`,
);
}
}
}

```

Powyższy kod odpowiada za pomiar czasu wykonania operacji bazodanowych. Metoda *measure* przyjmuje nazwę operacji oraz funkcję wykonującą zapytanie. Po zakończeniu działania wypisuje czas w milisekundach, co pozwala porównywać wydajność PostgreSQL i MongoDB.

10. Repozytorium jako warstwa dostępu do danych

```

async findByFirebaseUid(firebaseUid) {
  return DatabaseQueryTimer.measure("User.findUnique", () =>
    this.prisma.user.findUnique({
      where: { firebaseUid },
    }),
  );
}

```

Powyższy kod przedstawia przykład metody repozytorium odpowiedzialnej za pobranie użytkownika po identyfikatorze Firebase. Zapytanie jest wykonywane przez Prisma Client, a jego czas jest mierzony za pomocą *DatabaseQueryTimer*. Dzięki zastosowaniu repozytoriów logika biznesowa nie musi bezpośrednio znać szczegółów komunikacji z bazą danych.

Instrukcja uruchomienia

1. Pobranie projektu z Githuba:

git clone <https://github.com/Benonki/ToDo-app.git>

2. Uruchomienie Backendu na jeden z 2 sposobów:

a. Sposób a - MongoDB:

Zainstalowanie zależności:

```
npm i
```

Utworzenie i ustawienie danych w pliku .env (wartości przykładowe znajdują się w .env.example).

Utworzenie pliku serviceAccount.json, który jest potrzebny do działania logowania Firebase'a .

Odpalenie serwera (należy wpisać w konsoli):

```
node server.js
```

b. Sposób b - PostgreSQL:

Zainstalowanie zależności:

```
npm i
```

Utworzenie i ustawienie danych w pliku .env (wartości przykładowe znajdują się w .env.example).

Utworzenie pliku serviceAccount.json, który jest potrzebny do działania logowania Firebase'a .

Odpalenie serwera (należy wpisać w konsoli):

```
npm run dev
```

3. Uruchomienie Frontendu (należy wpisywać w konsoli):

Przejsć do katalogu Frontend:

```
cd Frontend
```

Zainstalowanie zależności:

```
npm i
```

W przypadku informacji o błędach:

```
npm audit fix
```

Po zainstalowaniu należy uruchomić aplikację:

```
npm run dev
```

Po poprawnym uruchomieniu frontend będzie dostępny w przeglądarce pod adresem:

```
http://localhost:5173/
```

Charakterystyka formatu JSON

Współczesne aplikacje bardzo często wymieniają i przechowują dane w formacie JSON, który jest lekkim formatem zapisu danych opierającym się na parach klucz-wartość.

Dane zapisane w formacie JSON mogą zawierać obiekty, tablice, tekst, liczby, wartości logiczne oraz wartości puste. Dzięki temu JSON dobrze sprawdza się przy reprezentowaniu danych o strukturze zagnieżdżonej, na przykład konfiguracji, odpowiedzi API, profili użytkowników lub list zadań.

Przykładowy dokument JSON może wyglądać następująco:

```
{
  "title": "Spotkanie",
  "startTime": "2026-06-03T10:00:00.000Z",
  "endTime": "2026-06-03T11:00:00.000Z",
  "color": "#4A90E2",
  "tags": ["praca", "ważne"]
}
```

W PostgreSQL dane mogą być przechowywane w polach typu JSON lub JSONB, natomiast MongoDB naturalnie zapisuje dane w formie dokumentów zbliżonych do JSON. Dzięki temu możliwe jest porównanie podejścia relacyjnego z podejściem dokumentowym pod względem wygody implementacji, struktury danych oraz wydajności operacji bazodanowych.

Porównanie MongoDB a PostgreSQL

- Model danych

Pierwszą różnicą jest sposób reprezentacji danych. W relacyjnej bazie danych informacje są przechowywane w tabelach składających się z kolumn i wierszy. Każda tabela ma z góry określoną strukturę, a powiązania między tabelami są realizowane przez klucze główne i klucze obce. Taki model jest uporządkowany i dobrze sprawdza się tam, gdzie dane mają stałą strukturę oraz występują liczne zależności między obiektami.

W bazach dokumentowych dane są zapisywane jako dokumenty podobne do JSON. Dokument może zawierać pola proste, obiekty zagnieżdżone oraz tablice. Dzięki temu dane, które w aplikacji są używane razem, mogą być zapisane razem w jednym dokumencie. Jest to szczególnie wygodne w sytuacji, gdy aplikacja często pobiera cały obiekt, na przykład użytkownika razem z listą jego zadań dla danego dnia.

PostgreSQL z typem JSONB stanowi rozwiązanie pośrednie. Z jednej strony nadal pozwala korzystać z tabel, relacji i języka SQL, a z drugiej umożliwia zapis bardziej elastycznych struktur dokumentowych w wybranych kolumnach. Oznacza to, że część danych może być modelowana relacyjnie, a część jako JSON.

- Relacja jeden do wielu

Relacja jeden do wielu została w projekcie przedstawiona na przykładzie użytkownika posiadającego wiele zadań. W klasycznym modelu relacyjnym każdy użytkownik jest zapisany w tabeli User, natomiast każde zadanie jest osobnym rekordem w tabeli TaskItem. Zadanie posiada pole userId, które wskazuje właściciela zadania. Dzięki temu baza danych może wymusić poprawność relacji i uniemożliwić zapis zadania przypisanego do nieistniejącego użytkownika.

W wariancie PostgreSQL z JSONB możliwe jest przechowywanie wielu zadań w jednym polu JSON. Przykładowo jeden rekord może zawierać datę, identyfikator użytkownika oraz tablicę zadań zapisanych jako JSON. Takie podejście upraszcza pobieranie wszystkich zadań z jednego dnia, ponieważ aplikacja otrzymuje od razu cały zestaw danych. Jednocześnie utrudnia to wykonywanie operacji na pojedynczych elementach tablicy, takich jak edycja jednego zadania, filtrowanie po wybranym polu lub dokładne indeksowanie każdego elementu.

W MongoDB możliwe są dwa warianty. Pierwszy polega na zagnieżdżeniu tablicy zadań w jednym dokumencie, na przykład dokumencie reprezentującym zadania użytkownika z konkretnego dnia. Drugi polega na zapisaniu każdego zadania jako osobnego dokumentu z polem userId. Wariant zagnieżdżony jest wygodny przy częstym pobieraniu całego dnia zadań, natomiast wariant referencyjny jest wygodniejszy, gdy zadania są często edytowane pojedynczo lub wyszukiwane niezależnie.

- Relacja wiele do wielu

Relacja wiele do wielu jest jednym z obszarów, w których różnice między modelem relacyjnym a dokumentowym są szczególnie widoczne. W analizowanej aplikacji do zarządzania zadaniami taką relację przedstawiono na przykładzie zadań i tagów. Jedno zadanie może posiadać wiele tagów, np. „studia”, „ważne” i „projekt”, a jeden tag może być przypisany do wielu różnych zadań.

W klasycznym modelu relacyjnym PostgreSQL taka relacja wymaga utworzenia trzech tabel: tabeli `task`, tabeli `tag` oraz tabeli pośredniczącej `task\_tag`. Tabela `task` przechowuje dane zadania, tabela `tag` przechowuje dostępne tagi, natomiast tabela `task\_tag` zapisuje powiązania między zadaniami i tagami poprzez identyfikatory `task\_id` oraz `tag\_id`. Dzięki zastosowaniu kluczy obcych baza danych może kontrolować poprawność relacji, na przykład uniemożliwić przypisanie zadania do tagu, który nie istnieje.

W MongoDB w projekcie zastosowano podejście dokumentowe z referencjami. Tagi nie są przechowywane bezpośrednio jako zwykłe teksty w dokumencie zadania, lecz znajdują się w osobnej kolekcji Tag. Każdy tag posiada własny identyfikator oraz nazwę, np. „studia”, „ważne” lub „projekt”. W dokumencie zadania przechowywana jest natomiast tablica identyfikatorów tagów, np. tags: [ObjectId("..."), ObjectId("...")]. Oznacza to, że jedno zadanie może być powiązane z wieloma tagami, a jeden tag może być wykorzystywany przez wiele różnych zadań.

Podczas dodawania lub edycji zadania aplikacja przekazuje listę nazw tagów do backendu. Backend sprawdza, czy dany tag istnieje już w kolekcji Tag. Jeżeli tag istnieje, do zadania przypisywany jest jego identyfikator. Jeżeli tag nie istnieje, zostaje najpierw utworzony jako nowy dokument w kolekcji Tag, a następnie jego identyfikator jest zapisywany w tablicy tagów zadania. Dzięki temu ograniczone zostaje powielanie tych samych tagów w wielu dokumentach.

- Zapytania i spójność danych

PostgreSQL wykorzystuje język SQL, który dobrze sprawdza się przy danych tabelarycznych i relacyjnych. Umożliwia wykonywanie operacji JOIN, filtrowanie, sortowanie oraz agregowanie danych. Pozwala także wykonywać zapytania po kolumnach JSONB, jednak są one zwykle mniej czytelne niż klasyczne zapytania do zwykłych kolumn.

MongoDB opiera zapytania na strukturze dokumentu. Jeżeli dane są zapisane tak, jak aplikacja je odczytuje, pobranie informacji może być bardzo proste, ponieważ cała potrzebna struktura znajduje się w jednym dokumencie. Problem pojawia się przy bardziej złożonych relacjach, gdy trzeba łączyć dane z wielu kolekcji. MongoDB również obsługuje indeksy i transakcje, ale przy relacjach między dokumentami więcej kontroli musi wykonywać aplikacja. Trzeba samodzielnie sprawdzać, czy wskazane identyfikatory dokumentów rzeczywiście istnieją.

Pod względem spójności PostgreSQL ma przewagę, ponieważ wykorzystuje klucze obce, ograniczenia i transakcje. Dzięki temu może wymusić, aby zadanie należało do istniejącego użytkownika, a po usunięciu użytkownika automatycznie usunąć jego zadania.

Testy

- Założenia testowe

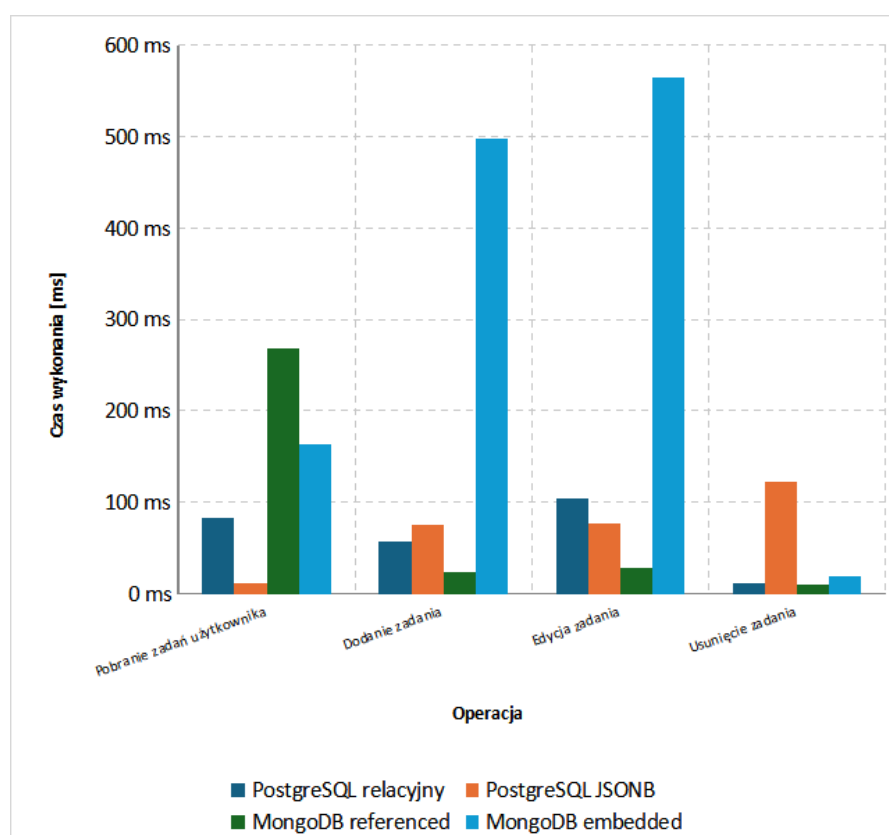
- Każdy test należy wykonać na takim samym zestawie danych (1440 zadań dla jednego dnia).
- Dla każdego testu należy zapisać czas wykonania.
- Każdy test należy wykonać na tym samym sprzęcie.

- Relacja jeden do wielu

Pierwszy test dotyczy relacji jeden do wielu, w której jeden użytkownik może posiadać wiele zadań. Jest to podstawowy przypadek użycia w analizowanym projekcie, ponieważ pozwala sprawdzić, czy lepiej przechowywać zadania jako osobne rekordy, jako tablicę JSONB, jako tablicę zagnieżdżoną w dokumencie MongoDB, czy jako osobne dokumenty z referencją do użytkownika.

Tabela 1. Wyniki testu relacji jeden do wielu

Operacja	PostgreSQL relacyjny	PostgreSQL JSONB	MongoDB referenced	MongoDB embedded
Pobranie zadań użytkownika	83.25 ms	11.85 ms	267.89 ms	163.88 ms
Dodanie zadania	57.41 ms	74.6 ms	24.07 ms	497.55 ms
Edycja zadania	103.65 ms	76.03 ms	27.46 ms	563.95 ms
Usunięcie zadania	11.34 ms	122.24 ms	9.14 ms	19.06 ms



Rysunek 8. Wyniki testu relacji jeden do wielu

Na podstawie wyników można zauważyć, że wydajność zależy głównie od sposobu modelowania danych, a nie tylko od rodzaju bazy. Modele, w których dane są przechowywane razem, mogą przyspieszać odczyt, ale utrudniać częste modyfikacje pojedynczych elementów. Widać to szczególnie w przypadku MongoDB embedded, gdzie operacje dodawania i edycji są kosztowne, ponieważ wymagają pracy na większym dokumencie.

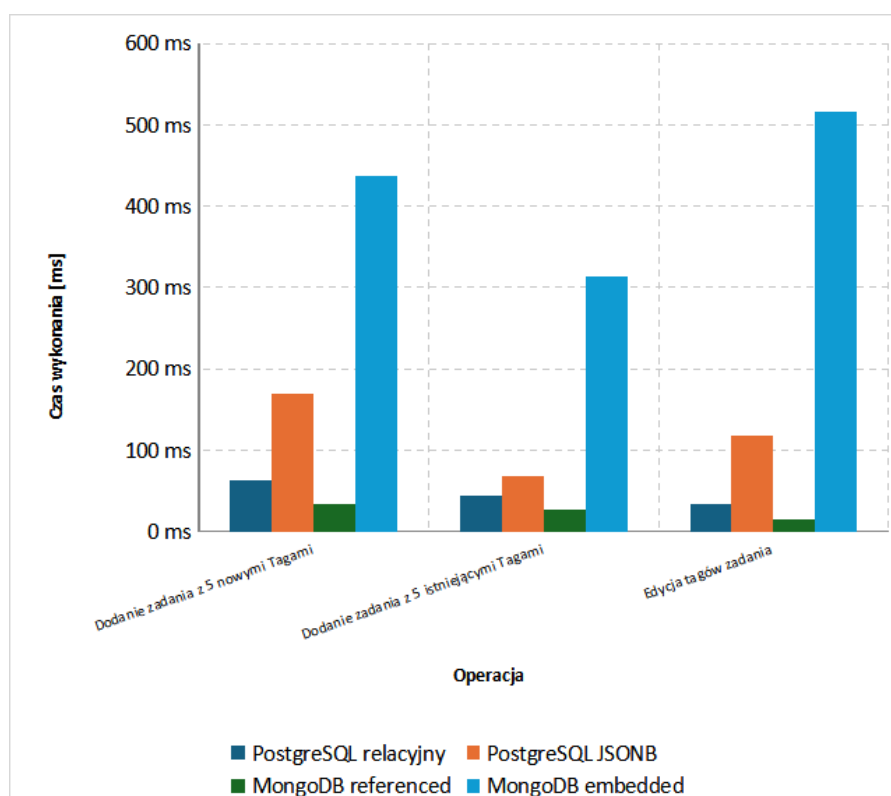
Modele oparte na osobnych rekordach lub referencjach lepiej sprawdzają się przy częstych zmianach danych, ponieważ operacje dotyczą mniejszych fragmentów. Test pokazuje więc, że wybór struktury bazy powinien zależeć od tego, czy aplikacja częściej pobiera całe zestawy danych, czy modyfikuje pojedyncze zadania.

- Relacja wiele do wielu

Drugi test dotyczy relacji wiele do wielu. W analizowanej aplikacji webowej do zarządzania zadaniami taki przypadek został przedstawiony na przykładzie zadań i tagów. Jedno zadanie może posiadać wiele tagów, np. „studia”, „ważne” lub „projekt”, a jeden tag może być przypisany do wielu różnych zadań.

Tabela 2. Wyniki testu relacji wiele do wielu

Operacja	PostgreSQL relacyjny	PostgreSQL JSONB	MongoDB referenced	MongoDB embedded
Dodanie zadania z 5 nowymi Tagami	62.65 ms	169.57 ms	33.44 ms	437.38 ms
Dodanie zadania z 5 istniejącymi Tagami	43.43 ms	67.09 ms	26.41 ms	313.35 ms
Edycja tagów zadania	33.81 ms	118.1 ms	14.21 ms	516.84 ms



Rysunek 9. Wyniki testu relacji wiele do wielu

W przypadku relacji wiele-do-wielu szczególnie widoczny jest wpływ sposobu przechowywania powiązań między zadaniami i tagami. Modele oparte na referencjach lub tabeli pośredniej lepiej radzą sobie z aktualizacją relacji, ponieważ zmiana dotyczy konkretnych rekordów lub dokumentów. Natomiast model zagnieżdżony wymaga modyfikacji większej struktury danych, dlatego przy dodawaniu i edycji tagów może generować większy narzut.

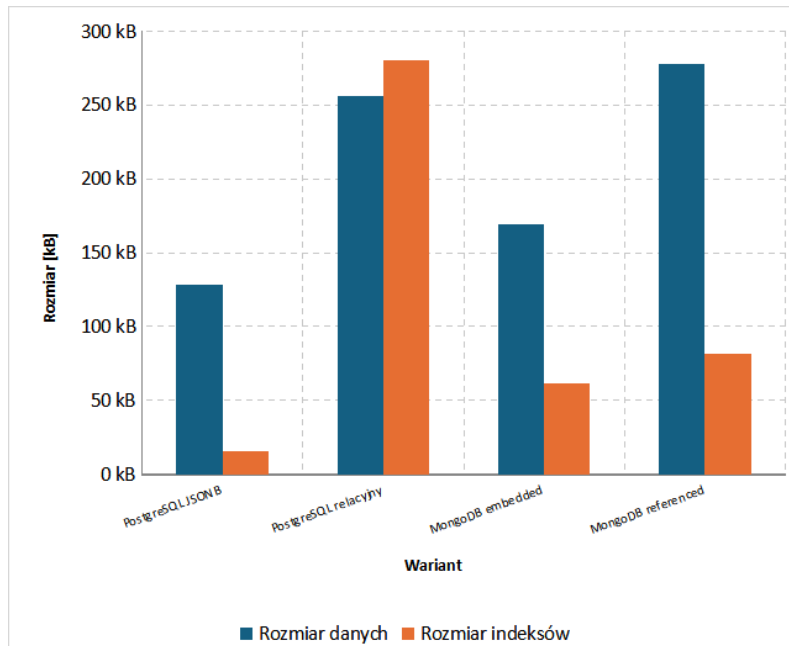
Wyniki pokazują, że relacja wiele-do-wielu jest mniej korzystna dla struktur silnie zagnieżdżonych, zwłaszcza gdy powiązania często się zmieniają. W takim przypadku bardziej praktyczne są modele, które przechowują zadania i tagi oddzielnie, a relacje między nimi zapisują osobno.

- Rozmiar danych i indeksów

Kolejny test pokazuje, ile miejsca zajmują dane oraz indeksy w poszczególnych wariantach. Jest to istotne, ponieważ model dokumentowy może ograniczać liczbę dokumentów, ale jednocześnie w przypadku denormalizacji może prowadzić do powtarzania tych samych informacji w wielu miejscach. Model relacyjny zwykle tworzy więcej tabel i rekordów, ale ogranicza redundancję danych.

Tabela 3. Porównanie rozmiaru danych i indeksów

Wariant	Liczba rekordów/ dokumentów	Rozmiar danych	Rozmiar indeksów
PostgreSQL JSONB	1	128 kB	16 kB
PostgreSQL relacyjny	1440	256 kB	280 kB
MongoDB embedded	1	168.89 kB	61.44 kB
MongoDB referenced	1440	277.98 kB	81.92 kB



Rysunek 10. Porównanie rozmiaru danych i indeksów

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
referencedtasks	-	57.34 kB	277.92 kB	1.4K	193.00 B	3	81.92 kB
tags	-	36.86 kB	569.00 B	5	113.00 B	2	65.54 kB
tasks	-	49.15 kB	168.89 kB	1	168.89 kB	3	61.44 kB
users	-	32.77 kB	437.00 B	2	218.00 B	2	73.73 kB

Rysunek 11. Wielkość zajmowanej pamięci - MongoDB

	AZ schemat	AZ tabela	AZ rozmiar_danych	AZ rozmiar_indeksow	AZ rozmiar_calkowity
1	public	TaskItem	256 kB	280 kB	536 kB
2	public	Task	128 kB	16 kB	144 kB
3	public	User	16 kB	48 kB	64 kB
4	public	_prisma_migrations	16 kB	16 kB	32 kB
5	public	Tag	8192 bytes	16 kB	24 kB
6	public	_TagToTaskItem	8192 bytes	16 kB	24 kB

Rysunek 12. Wielkość zajmowanej pamięci - PostgreSQL

Wyniki pokazują, że w badanym przypadku modele zagnieżdżone zajmują mniej miejsca niż modele relacyjne i referencyjne. PostgreSQL JSONB oraz MongoDB embedded przechowują dane w mniejszej liczbie rekordów lub dokumentów, co ogranicza narzut związany z wieloma osobnymi wpisami oraz indeksami. Szczególnie widoczne jest to w PostgreSQL relacyjnym, gdzie duża liczba rekordów i relacji powoduje znaczny wzrost rozmiaru indeksów. Oznacza to, że modele zagnieżdżone mogą być korzystniejsze pod względem wykorzystania pamięci, jednak wcześniejsze testy pokazują, że nie zawsze przekłada się to na lepszą wydajność operacji modyfikujących dane.

Wygoda i czytelność zapytań

Oprócz samej wydajności ważna jest również wygoda pisania zapytań. W tym teście porównano, jak wygląda zapytanie pobierające zadania z jednego dnia, zapytanie pobierające jedno zadanie po identyfikatorze, zapytanie sortujące zadania po czasie rozpoczęcia oraz zapytanie pobierające dane przez relację wiele do wielu.

Tabela 4. Porównanie wygody i czytelności zapytań

Operacja	PostgreSQL relacyjny	PostgreSQL JSONB	MongoDB embedded	MongoDB referenced
Pobranie zadań z jednego dnia	Wysoka proste <i>WHERE</i>	Średnia filtrowanie w JSONB	Średnia filtrowanie tablicy w dokumencie	Wysoka proste wyszukiwanie po polach
Pobranie zadania po identyfikatorze	Wysoka <i>WHERE id</i>	Niska szukanie elementu w JSONB	Niska szukanie w zagnieżdżonej tablicy	Wysoka <i>find</i> po <i>_id</i>
Sortowanie po czasie rozpoczęcia	Wysoka <i>ORDER BY</i>	Średnia sortowanie po polu z JSONB	Średnia często wymaga agregacji lub sortowania w aplikacji	Wysoka sortowanie po polu dokumentu
Pobieranie danych przez relację wiele do wielu	Średnia wymagane <i>JOIN</i>	Niska mniej czytelne operacje na JSONB	Średnia proste przy odczycie, trudniejsze przy zmianach	Średnia wymaga referencji lub <i>\$lookup</i>

W zakresie wygody pisania zapytań najbardziej czytelne są modele, w których zadania są przechowywane jako osobne rekordy lub dokumenty. Dzięki temu pobieranie po identyfikatorze, filtrowanie po dacie oraz sortowanie po czasie rozpoczęcia można zapisać w prosty i naturalny sposób.

Modele JSONB i embedded są wygodne, gdy pobierany jest cały dokument lub cały zestaw danych użytkownika. Stają się jednak mniej czytelne przy operacjach na pojedynczych elementach zagnieżdżonej struktury, ponieważ wymagają odwoływania się do pól wewnątrz JSON-a lub tablicy dokumentów.

W przypadku relacji wiele do wielu model relacyjny jest bardziej formalny i wymaga użycia tabeli pośredniej oraz JOIN, ale pozostaje czytelny i uporządkowany. W modelach dokumentowych takie relacje są mniej naturalne, szczególnie gdy tagi mają być współdzielone przez wiele zadań i często aktualizowane.

- Spójność danych

Test powinien sprawdzić, w jakim stopniu baza danych samodzielnie pilnuje poprawności relacji i czy aplikacja musi wykonywać dodatkowe kontrole. W modelu relacyjnym PostgreSQL można wykorzystać klucze obce, ograniczenia i transakcje. W MongoDB relacje oparte na identyfikatorach dokumentów wymagają zwykle większej kontroli po stronie aplikacji, szczególnie wtedy, gdy dane są rozdzielone między kilka kolekcji.

Tabela 5. Wyniki testu spójności danych

Operacja	PostgreSQL relacyjny	PostgreSQL JSONB	MongoDB embedded	MongoDB referenced
Dodanie zadania dla nieistniejącego użytkownika	Blokowane przez klucz obcy	Częściowo kontrolowane, zależy od klucza obcego na <i>userId</i>	Wymaga kontroli aplikacji	Wymaga kontroli aplikacji
Dodanie niepoprawnego powiązania wiele do wielu	Blokowane przez klucze obce w tabeli pośredniej	Wymaga kontroli aplikacji	Wymaga kontroli aplikacji	Wymaga kontroli aplikacji lub transakcji
Usunięcie użytkownika	Kontrolowane przez bazę, np. <i>CASCADE</i> lub <i>RESTRICT</i>	Częściowo kontrolowane	Dane zagnieżdżone mogą zostać usunięte razem z dokumentem	Wymaga usunięcia lub aktualizacji powiązanych dokumentów
Usunięcie elementu powiązanego	Kontrolowane przez relacje i ograniczenia	Wymaga dodatkowej logiki dla danych w JSONB	Wymaga aktualizacji dokumentu zagnieżdżonego	Wymaga kontroli referencji między kolekcjami

Wyniki pokazują, że największą kontrolę nad spójnością danych zapewnia model relacyjny PostgreSQL, ponieważ baza może samodzielnie pilnować poprawności relacji za pomocą kluczy obcych, ograniczeń oraz reguł usuwania. Dzięki temu trudniej przypadkowo dodać zadanie dla nieistniejącego użytkownika albo utworzyć błędne powiązanie wiele-do-wielu.

W modelach JSONB i dokumentowych większa część odpowiedzialności przechodzi na aplikację. Dane mogą być przechowywane elastyczniej, ale baza nie zawsze automatycznie sprawdza poprawność relacji ukrytych wewnątrz dokumentów lub tablic. MongoDB embedded ogranicza problem osieroconych danych, ponieważ powiązane informacje są zapisane razem, ale przy bardziej złożonych relacjach nadal wymaga dodatkowej logiki. MongoDB referenced jest bardziej elastyczne, lecz wymaga największej kontroli po stronie aplikacji, ponieważ same referencje między dokumentami nie działają tak jak klucze obce w bazie relacyjnej.

Podział pracy

- Benjamin Żak - Backend MongoDB, część wspólna Frontendu
- Jakub Maliński - Backend PostgreSQL, część wspólna Frontendu

Wnioski

Na podstawie przeprowadzonego porównania można stwierdzić, że zarówno relacyjne bazy danych, jak i składy dokumentowe mogą obsługiwać dane o strukturze JSON, jednak robią to w odmienny sposób. PostgreSQL pozwala połączyć klasyczny model relacyjny z możliwością przechowywania bardziej elastycznych struktur w polach JSON lub JSONB. MongoDB natomiast traktuje dokument jako podstawową jednostkę danych, dlatego naturalnie pasuje do przypadków, w których aplikacja operuje na całych dokumentach, obiektach zagnieżdżonych oraz danych o zmiennej strukturze.

PostgreSQL lepiej sprawdza się w sytuacjach, w których najważniejsza jest spójność danych, poprawność relacji oraz możliwość wykonywania złożonych zapytań łączących dane z wielu tabel. Klucze obce, transakcje i mechanizmy kontroli integralności sprawiają, że baza może sama blokować wiele niepoprawnych operacji. Jest to szczególnie istotne w systemach, w których dane są silnie powiązane i nie można dopuścić do powstawania osieroconych rekordów lub błędnych relacji.

MongoDB jest korzystnym rozwiązaniem wtedy, gdy dane są naturalnie dokumentowe i często odczytywane jako całość. Zagnieżdżenie danych w jednym dokumencie może uprościć pobieranie informacji oraz ograniczyć liczbę zapytań wykonywanych przez aplikację. Jednocześnie takie podejście wymaga ostrożności, ponieważ nadmierna denormalizacja może prowadzić do powielania danych i utrudniać ich późniejszą aktualizację.

Wariant PostgreSQL JSONB można uznać za rozwiązanie pośrednie. Umożliwia on przechowywanie danych dokumentowych wewnątrz bazy relacyjnej, ale nie zawsze zastępuje dobrze zaprojektowany model tabelaryczny. JSONB jest szczególnie przydatny dla danych dodatkowych, konfiguracyjnych, częściowo zmiennych lub takich, których struktura nie musi być zawsze identyczna. W przypadku danych o wyraźnych relacjach i częstych zapytaniach przekrojowych klasyczne tabele nadal pozostają bardziej czytelnym i bezpiecznym rozwiązaniem.

Ostateczny wybór technologii powinien zależeć od charakteru danych oraz sposobu korzystania z nich przez aplikację. Jeśli aplikacja często pobiera jeden kompletny dokument i rzadko wykonuje skomplikowane relacje między wieloma zbiorami danych, MongoDB może być wygodniejsze. Jeżeli natomiast system wymaga silnych relacji, transakcji, dokładnej kontroli spójności i rozbudowanych zapytań, lepszym wyborem będzie PostgreSQL. W praktyce oba podejścia mogą być poprawne, ale wymagają innego sposobu projektowania struktury danych.